

Grado: Ingeniería Electrónica de Comunicaciones
Asignatura: Procesamiento de Señales
Curso: 2025/26

Práctica 1
Introducción: señales y sistemas

Objetivos

- Aprender a generar, representar, manipular y analizar diferentes tipos de señales con ayuda de Matlab.

Generación, representación y análisis de señales

La generación, representación y el análisis de señales mediante computador son tareas fundamentales dentro del procesamiento de señales. En esta práctica vamos a trabajar con señales de diferentes tipos: continuas y discretas, con simetría (par/impar) o no, periódicas y aperiódicas. Un sistema de procesamiento digital (en este caso el PC) es únicamente capaz de representar y operar sobre señales digitales, es decir, aquellas que son discretas en el tiempo y cuantizadas[1]. Por tanto, para manipular señales analógicas, deberemos realizar previamente una conversión de la señal (a través de un proceso de muestreo y cuantización que estudiaremos en el tema 3), o bien trabajar de forma simbólica con la expresión matemática de la señal. En esta introducción vamos a ver una serie de conceptos y herramientas que nos van a permitir trabajar con señales en Matlab.

Definición de Variables

Una variable es un nombre simbólico que hace referencia a un espacio de memoria donde se almacena un valor. Para definir una variable en Matlab basta con asignarle un valor:

```
x = 1;
y = 3.2;
```

[1] Aunque no vamos a considerar aún el efecto de la cuantización, ésta conlleva una pérdida irreversible de informacíón. Además, tener una resolución y longitud de palabra finita puede tener consecuencias de mayor o menor importancia, y como veremos más adelante puede incluso provocar problemas de inestabilidad según el tipo de filtro y método de diseño que se utilice.

El tipo básico de variable es una matriz, de manera que un escalar es simplemente una matriz de $1x1$ elementos. Para inicializar una matriz con sus elementos podemos indicarlos entre corchetes, separando por espacios o ',' los elementos de un fila, y las columnas mediante ';' o saltos de línea. Las siguientes definiciones (entre otras) son equivalentes:

```
x = [0, 1, 2; 1, 1, 1];
x = [
    0 1 2
    1 1 1
]
```

También podemos generar una rejilla de puntos linealmente espaciados mediante una expresión del tipo: `x=[start:step:end]` o `x=linspace(start, end, n)`. Para obtener una rejilla de puntos logarítmicamente espaciados tenemos `logspace(a, b, n)`, lo que generará valores n entre 10^a y 10^b.

Operaciones básicas

Para operar con variables escalares disponemos de los operadores aritméticos habituales ($*, /, +, -, \hat{}, ...$).

```
z = (x + 3*y) / 2;
```

Estos operadores también se aplican a vectores/matrices igual que en el caso escalar, aunque con algunos matices a tener en cuenta. En primer lugar, para poder realizar una operación entre vectores/matrices las dimensiones deben ser compatibles[2]. Para la mayoría de operadores binarios, dos matrices (multidimensionales) se consideran compatibles si, para cada dimensión, el número de elementos es el mismo o bien uno de ellos es un escalar. La siguiente operación es correcta:

```
>> x = [0, 0.2, 1, 5, -2];
>> x + 2*x
ans =
    0.0000    0.6000    3.0000   15.0000   -6.0000
>> 2 * x
ans =
         0.0000    0.4000    2.0000   10.0000   -20.0000
>> x + 1
ans =
    1.0000    1.2000    2.0000    6.0000   -1.0000
```

En este ejemplo, la operación `2*x` es válida a pesar de que los factores 2 y `x` no tengan la misma dimensión, ya que uno de los elementos es escalar. Igual para `x+1`. Si las dimensiones son compatibles (aunque no sean iguales, Matlab expandirá automáticamente según sea necesario) la operación se realizará correctamente. Sin embargo, la operación `x + y` no es válida ya que la variable `x` contiene un número de elementos diferente al de `y`, por lo que el intento de realizar la suma arroja un error.

```
>> y = [1:10];
>> x + y
```

[2] https://es.mathworks.com/help/matlab/matlab_prog/compatible-array-sizes-for-basic-operations.html?lang=en



```
ans =
Matrix dimensions must agree.
```

En segundo lugar, para algunas operaciones tenemos dos posibilidades diferentes: operar siguiendo las reglas habituales de las operación con matrices ($*, /, \hat{}$), o realizar las operaciones elemento por elemento (*element-wise*) anteponiendo un punto al operador ($.*, ./, .\hat{}$). Ambas operaciones son útiles y tendrán o no sentido según el contexto. Es importante seleccionar el operador adecuado, ya que en caso contrario ocurrirá un error debido a la incompatibilidad de las dimensiones o, peor aún, la operación no arrojará ningún error pero el resultado no será el esperado.

```
>> A = eye(2);
>> B = [1; 2];
>> A * B
ans =
        1
        2
>> A .* B
ans =
        1     0
        0     2
```

En este ejemplo, tanto `A*B` (producto matricial) como `A.*B` (elemento a elemento) están definidos, pero el resultado es diferente (en el primer caso un vector columna, en el segundo una matriz).

Definición de una señal

Para definir una señal en Matlab podemos utilizar diferentes enfoques. Vamos a describir algunas de las herramientas o conceptos que utilizaremos para tal fin.

Series temporales

Una serie temporal es un vector de datos muestreados en el tiempo (habitualmente con un periodo de muestreo fijo). Por ejemplo, podemos definir una secuencia de valores directamente con su vector de tiempos asociado:

```
>> t = [0, 1, 2, 3, 4, ...];
>> x = [5, 1.5, -4, 1.2, ...];
```

O bien, si conocemos la expresión matemática de la señal:

```
>> N = 5;
>> n = [0:N-1];
>> x = sin(n)
x =
    0    0.8415    0.9093    0.1411   -0.7568
```

donde, ademas de los valores que toma la señal (`x`), definimos los tiempos asociados a cada muestra (`n`). Si la señal es continua, podemos utilizar el mismo enfoque para representar una versión muestreada:

```
>> T = 5;
>> Ts = 0.1;
```

```
>> t = [0:Ts:T];
>> x = sin(t)
x =Columns 1 through 8
    0     0.0998     0.1987     0.2955     0.3894  ...
```

y los tiempos de muestreo están almacenados en `t`. En ambos casos, es importante remarcar que la señal esta definida no solo por los valores de la secuencia `x`, sino también por los tiempos en los que se evalúa (sobre todo para la señal continua, ya que en la discreta se puede asumir $n \in (0, N-1)$).

Un inconveniente de definir la señal de esta manera es que, aunque evidentemente existe una relación entre `t` y `x`, no hay ningún vínculo explícito entre ellas en el código. Otra forma más sistemática de representar la serie temporal es mediante una matriz, donde la primera columna representa el tiempo y la segunda columna el valor de la señal. De esta manera la relación entre t y x se hace explícita en la propia estructura de los datos. Además, es inmediata la generalización a varias señales, simplemente añadiendo más columnas a la tabla (deben compartir el vector de tiempos). Este formato es habitual por su simplicidad y flexibilidad, y se puede utilizar fácilmente para almacenar datos en un archivo de texto. Por ejemplo, para cargar un archivo 'datos.txt':

```
'datos.txt'
0.0  1.0
0.1  0.2
0.2  0.4
0.3  -0.4
>> datos = load('datos.txt')
datos =
         0    1.0000
    0.1000    1.4000
    0.2000    0.4000
    0.3000   -0.4000
```

Un método conceptualmente similar pero algo más sofisticado es la utilización de un objeto `timeseries`, que permite almacenar información extra sobre la serie temporal.

```
>> ts = timeseries([1, 1.4, 0.4, -0,4]', [0.0, 0.1, 0.2, 0.3], 'name', 'Datos')
timeseries

Common Properties:
            Name: 'Datos'
            Time: [4x1 double]
        TimeInfo: [1x1 tsdata.timemetadata]
            Data: [4x1 double]
        DataInfo: [1x1 tsdata.datametadata]

More properties, Methods
```

Podemos acceder al vector de tiempos y a los datos a través de las propiedades del objeto:

```
>> ts.Time
ans =
```

```
      1
2      3
      4
      5

>> ts.Data
ans =

0.4132
0.3281
0.5691
0.2219
0.7151
```

Además, el objeto timeseries proporciona una serie de funcionalidades adicionales para trabajar con series temporales, por ejemplo:

```
>> ts.plot() % Representa graficamente
>> tsout = resample(ts, 1 + 0.1*[1:5]); % Remuestrea la serie temporal
```

Estas series temporales se pueden utilizar directamente en el analizador de señales de Matlab, una herramienta gráfica que permite, entre otras cosas, representar y explorar señales, trocear en partes, estimación del espectro, etc.[3]

Funciones

En general las funciones son útiles para encapsular código y hacerlo reutilizable. Un uso adecuado de las funciones puede facilitar en gran medida el trabajo, de manera que una vez resuelto un problema no sea necesario repetir el mismo código una y otra vez[4].

La siguiente sintáxis permite definir una función que recibe unos argumentos de entrada, realiza un procesamiento y devuelve unas salidas:

```
function [y1, ..., yN] = f(x1, ..., xM)
  % Hacer algo util (o no) con x1, ..., xM
  % Devolver salidas
  y1 = ...
  ...
  yM = ...
end
```

Las definiciones de función deben ir al final del archivo que contiene el *script* que estamos ejecutando, o bien en un archivo aparte que contenga únicamente funciones[5].

En el contexto de la generación de señales, podemos utilizar una función, entre otras cosas, para definir una señal sin tener que evaluarla inmediatamente en un intervalo temporal:

[3] Para abrir el analizador hay que escribir `signalAnalyzer`. Se puede utilizar para comparar con los resultados de la práctica.
[4] https://es.wikipedia.org/wiki/No_te_repitas
[5] https://es.mathworks.com/help/matlab/ref/function.html?lang=en



```
function y = f(t)
y = sin(t)end
```

O, utilizando una alternativa más compacta a la anterior definición mediante el operador @:

```
f = @(t) sin(t);
```

Esto puede resultar útil por ejemplo si queremos realizar operaciones en la variable independiente:

```
t = [-pi:0.1:pi];
y1 = f(t);
y2 = f(4*t);
```

Cálculo Simbólico

Matlab ofrece la posibilidad es trabajar de forma simbólica. Así, podemos definir la variable dependiente (p.ej. t) de tipo `syms` y, a continuación, definir la señal directamente como función de t:

```
syms t
x = sin(t) % Define una senal de forma simbolica. Para representar habra que evaluar
```

La primera instrucción indica a Matlab que t es una variable simbólica y debe tratarla como tal, mientras que la segunda declara x como una expresión que involucra la variable t, y por tanto también se tratará simbólicamente. Podemos manipular las expresiones mediante los operadores habituales (*, /, +, -, ...).

Además, el motor de cálculo simbólico proporciona una serie de funciones que nos permiten manipular las expresiones. Algunas de ellas se listan a continuación:

- `collect`, reordena una expresión en forma de polinomio en una variable v.
- `factor`, obtiene una factorización de una expresión: $factor(x^2 * y^2, x)$ devuelve $[y^2, x, x]$.
- `simplify`, busca una expresión equivalente pero mas simple: $simplify(sin(x)^2 + cos(x)^2)$ devuelve 1.
- `expand`, desarrolla una expresión mediante el producto de sus factores: $expand(sin(x + y))$ devuelve $sin(x) * cos(y) + cos(x) * sin(y)$

Más información sobre su uso y otras funciones en la ayuda de Matab.

Para representar gráficamente, primero debemos obtener una representación de la señal como una secuencia discreta. A continuación usaremos `plot` para graficar la secuencia. Para ello, substituimos la variable t por un vector de tiempos con el intervalo deseado:

```
>> Ts = 0.1;
>> t_intervalo = [0:Ts:10];
>> x_intervalo = subs(x_simbolica, t, t_intervalo);
```

Al substituir el vector de tiempos t, obtenemos otro vector del mismo tamaño que contiene la expresión (`x_simbolica`, en este caso) con la variable t reemplazada por los correspondientes valores numéricos. Este resultado lo podemos utilizar para representar la señal mediante `plot`.



```
>> plot(t_intervalo, x_intervalo)
```

Hay que destacar que, a pesar de la substitución, la expresión se sigue tratando como simbólica. Esto puede resultar apropiado si vamos a seguir operando de forma simbólica sobre el resultado. Si únicamente nos interesa tratar el resultado de forma numérica, es más eficiente convertir a una secuencia discreta mediante $eval$.

```
x_intervalo =
        [0, sin(1/5), sin(2/5), sin(3/5), sin(4/5), sin(1)]
>> x_secuencia = eval(x_intervalo)
x_secuencia =
        0    0.1987    0.3894    0.5646    0.7174    0.8415
```

Para apreciar mejor la diferencia, podemos comparar el tiempo de una multiplicación por un escalar para un vector de igual longitud de cada tipo:

```
>> tic; 2*x_intervalo; toc
Elapsed time is 0.002794 seconds.
>> tic; 2*x_secuencia; toc
Elapsed time is 0.000223 seconds.
```

En este caso, la diferencia es de un orden de magnitud para solo 100 elementos. Por este motivo, es importante elegir la representación más adecuada.

Análisis en frecuencia

Hasta ahora hemos visto como trabajar con señales en el dominio del tiempo. En algunos casos nos interesa pasar al dominio de la frecuencia, haciendo uso de difentes herramientas como el desarrollo en serie de Fourier, o las transformadas de Fourier, Laplace y Z. Al igual que en la discusión anterior, también existen diferentes enfoques, vamos a ver algunos de ellos.

Cálculo simbólico

Matlab proporciona varias funciones que permiten obtener de forma simbólica las transformadas de Fourier (`fourier,ifourier`), Laplace (`laplace,ilaplace`) y Z (`ztrans,iztrans`). Su aplicación es bastante sencilla en todos los casos, basta con indicar como parámetro la expresión (por supuesto, simbólica) a transformar:

```
>> syms n t
>> fourier(sin(t))
ans =

-pi*(dirac(w - 1) - dirac(w + 1))*1i
>> laplace(sin(t))
ans =

1/(s^2 + 1)
>> ztrans(sin(n))
ans =

(z*sin(1))/(z^2 - 2*cos(1)*z + 1)
```

Por defecto, `fourier` utiliza la definición bilateral ($t \in (-\infty, \infty)$), mientras que Laplace y Z siguen la definición unilateral ($t \in (0, \infty)$ y $n \in (0, \infty)$, respectivamente). Igualmente para la transformada inversa. Para obtener la transformada de Fourier de señales continuas, se utiliza `fourier`. También se puede aplicar `laplace` y a continuación utilizar el cambio de variable $s = jw$, siempre que la región de convergencia (ROC) contenga al eje imaginario. De la misma forma, para obtener la transformada de Fourier de señales discretas será posible aplicar `ztrans` y a continuación $z = e^{jw}$ si la ROC incluye a la circunferencia unidad. Otra opción es calcularla a mano a partir de tablas o de la definición de la transformada de Fourier.

En cualquier caso, para representar correctamente la transformada de Fourier de una señal, no hay que olvidar que en general dicha transformada es una señal compleja. Por tanto, es necesario mostrar tanto el módulo como la fase (o, de forma equivalente, la parte real e imaginaria). Con respecto al rango de frecuencias a mostrar, hablamos de espectro unilateral cuando representamos únicamente frecuencias positivas, y bilateral cuando representamos también las negativas. En el caso de una señal real, puesto que su transformada es una señal hermítica es frecuente mostrar solo el espectro unilateral, ya que la otra mitad puede ser inferida a partir de la primera.

Para señales puramente discretas o aquellas en las que no tenemos información sobre el periodo de muestreo, el rango de frecuencias habitualmente se elige $(-\pi, \pi)$ o $(0, 2\pi)$ para representar el espectro bilateral, o bien $(0, pi)$ para el unilateral. Si conocemos el periodo de muestreo, la relación entre la frecuencia discreta ω_d $(rad/muestra)$ y la continua ω_c (rad/s) es $\omega_c = \omega_d T_s$.



Tarea 1: Representa gráficamente las siguientes señales, utilizando un rango de valores de t (señales continuas) o n (señales discretas) adecuado para cada señal.

- $x_{a_1}(t) = \mathrm{e}^{-j15t}$
- $x_{a_2}(t) = \mathrm{e}^{2\pi t}$
- $x_{d_1}[n] = \cos(\frac{\pi n}{2}) + 2\cos(\pi n) + 0{,}3\cos(\frac{3\pi n}{2})$
- $x_{d_2}[n] = (-0{,}7)^n$

Ayuda: Para representar las señales continuas (dependientes de t) se utilizará el comando `plot` y un paso del vector de tiempo adecuado para que se vea la forma de la señal. Para representar las señales discretas (dependientes de n) se utilizará el comando `stem` (o un plot con puntos).

Pregunta: ¿Cómo has elegido el rango de valores para t y n?

Tarea 2: Realiza las siguientes operaciones con las señales definidas en la tarea 1. Representa gráficamente el resultado (una gráfica por cada operación).

- $y_{a_1}(t) = x_{a_1}(t) - x_{a_2}(t)$
- $y_{a_2}(t) = x_{a_1}(t)/x_{a_2}(t)$
- $y_{d_1}[n] = x_{d_1}[n] \cdot x_{d_2}[n]$
- $y_{d_2}[n] = 0{,}5\, x_{d_1}[n] + 2\, x_{d_2}[n]$

Pregunta: ¿Que otras operaciones se pueden realizar sobre las señales?

Tarea 3: Implementa una función que devuelva el resultado de realizar la **convolución discreta** entre dos señales genéricas `x1` y `x2`. Calcula $y_{d_3}[n] = (x_{d_1} * x_{d_2})[n]$, con las señales definidas en la tarea 1, utilizando la función `convolucion` desarrollada.

Ayuda: Utiliza la siguiente estructura.

```
function y = convolucion(x1, x2)
  % Aquí va el código que calcula la convolución
  y = ... % El resultado lo asignamos a la variable 'y'
end
```

Pregunta: ¿Que longitud tiene la señal resultado de la convolución, y_{d_3}?

Tarea 4: Utiliza la convolución $y[n] = \sum_{n=0}^{N} x[m]h[n-m]$ para calcular la respuesta del sistema LTI descrito por $h[n]$ cuando es excitado por la entrada $x[n]$. Representa en una gráfica las señales $h[n]$, $x[n]$ e $y[n]$.

$$h[n] = \begin{cases} 1/4 \ si \ 0 \leq n < 4 \\ 0 \text{ otro caso} \end{cases}$$

$$x[n] = \cos(\frac{\pi n}{5})\sin(\frac{2\pi n}{5}) \ 0 \leq n < 10$$

Pregunta: ¿Cómo has elegido el valor de N?

Tarea 5: (opcional) Calcula mediante `fft` y representa en una gráfica $H(\frac{2\pi k}{N})$, $X(\frac{2\pi k}{N})$ e $Y(\frac{2\pi k}{N})$, es decir, la *Transformada de Fourier Discreta* (DFT) de las señales $h[n]$, $x[n]$ e $y[n]$ definidas en la Tarea 4.

Pregunta: ¿Qué relación hay entre las DFT representadas y la Transformada de Fourier de Tiempo Discreto (DTFT) de las señales?

